

BRIEFR System Design

Version: 1.1 (beta)
Last updated: 2026-06-07
Source of truth: `/workspace` codebase at commit audited for v1.1

1. Overview

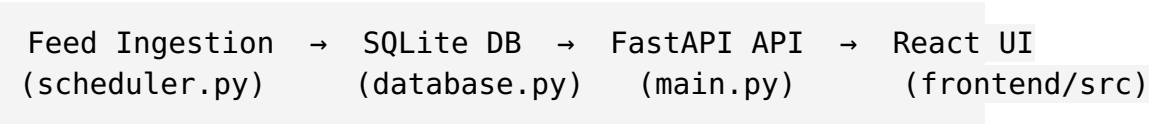
BRIEFR is a CVE intelligence platform that ingests vulnerability data from NVD, CISA KEV, EPSS, and MITRE sources into a local SQLite database, enriches records with threat-context feeds (OTX, Sploitus, GreyNoise, OSV, CIRCL), and presents them through a React analyst UI with IOC lookup, risk scoring, correlation, and PDF export.

It is built for security analysts, small security teams, and solo researchers who need a single-pane view of what is exploitable, what is in KEV, and what matches their stack — without standing up a full SIEM or commercial threat-intel platform.

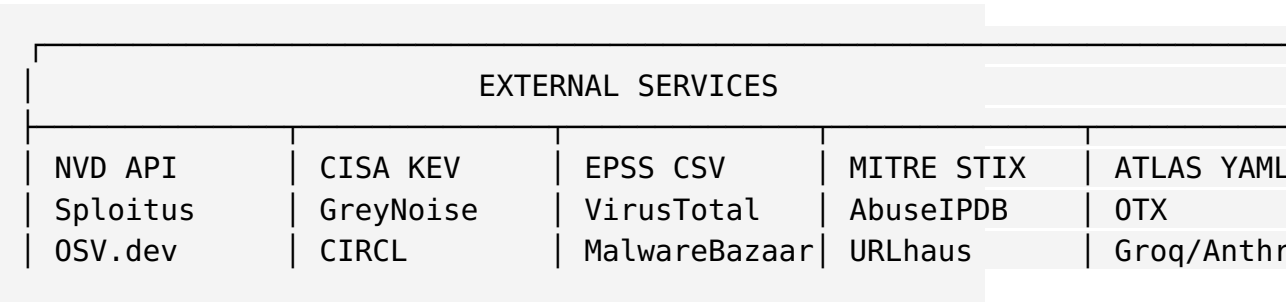
The core problem it solves is **analyst time**: aggregating scattered CVE metadata, exploitation signals, ATT&CK mapping, and IOC enrichment into one fast, dark-mode workflow that runs on a single server with optional API keys.

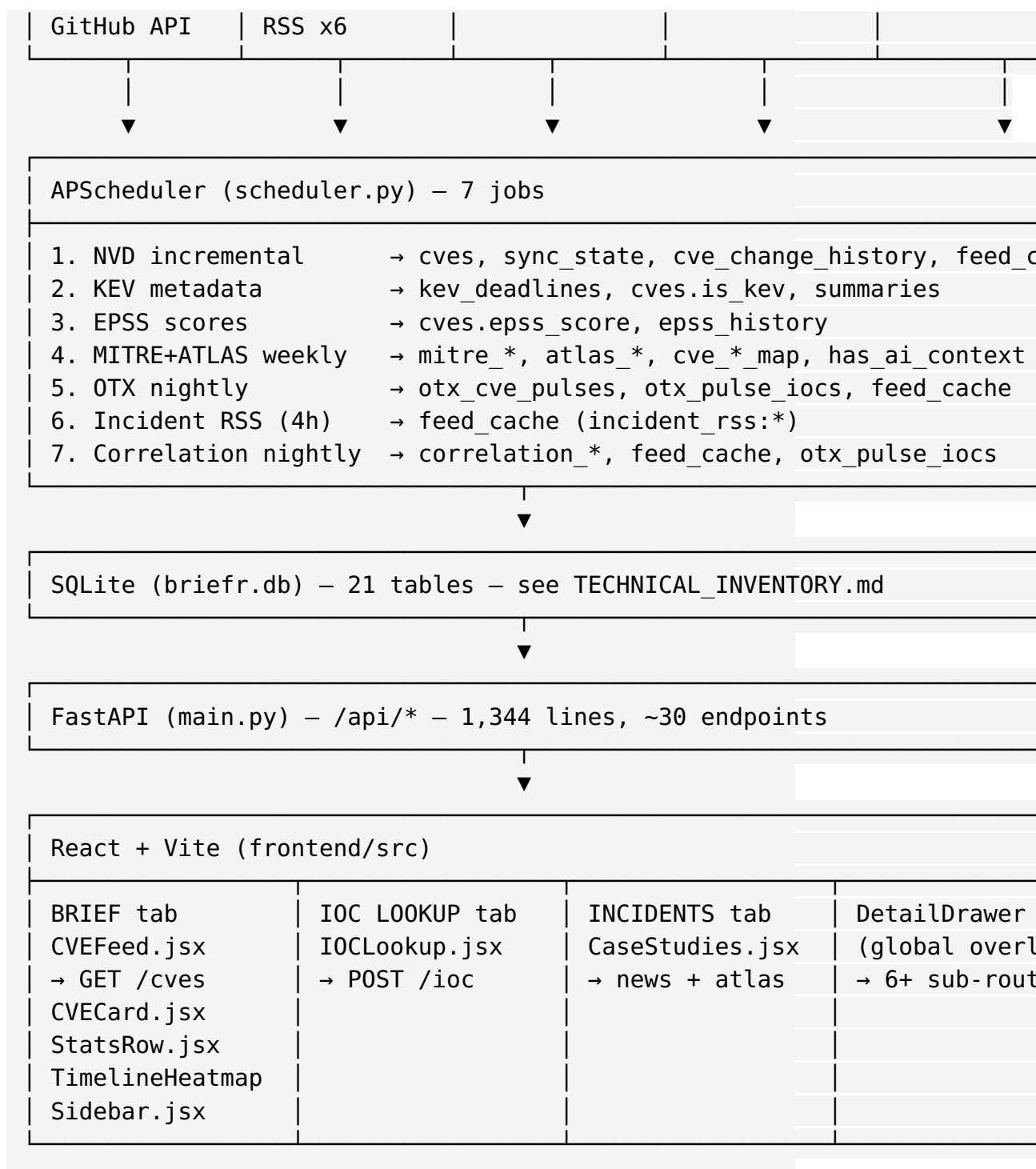
2. Architecture

Four-layer model



ASCII architecture diagram





Mermaid source: [docs/diagrams/architecture.mermaid](#)

DB tables → primary API readers

Table(s)	Primary endpoints	Frontend consumers
cves	GET /api/cves, GET /api/cves/{id},	CVEFeed, CVECard, DetailDrawer,

Table(s)	Primary endpoints	Frontend consumers
	GET /api/stats	StatsRow, TimelineHeatmap
kev_deadlines	GET /api/kev/deadlines , embedded in CVE detail	Sidebar, DetailDrawer sentences
epss_history	GET /api/cves/{id}/epss-history , momentum	DetailDrawer EPSS sparkline
mitre_techniques , cve_technique_map	GET /api/techniques/top , CVE techniques field	Sidebar, DetailDrawer Intel tab
atlas_* , cve_atlas_map	GET /api/atlas/* (not wired into CVE detail — known issue)	DrawerAtlasSection (expects fields missing from API)
otx_*	CVE detail, correlation, IOC lookup	DetailDrawer Intel tab, IOCLookup
feed_cache , ioc_cache	Internal — speeds enrichment	Transparent to UI
correlation_*	GET /api/cves/{id}/correlation	DetailDrawer correlation section
cve_exploits	Via Sploitius loader in CVE detail	DetailDrawer Intel tab
cve_change_history	GET /api/changes	— (API only)
api_usage	GET /api/usage , GET /api/usage/ioc	IOCLookup quota display

3. Data Flow

A. CVE lifecycle

1. **Ingest:** `scheduler.run_nvd_incremental_sync` → `feeds/nvd.py:fetch_nvd_cve_updates` (NVD REST 2.0, watermark in `sync_state`).
2. **Persist:** `database.upsert_cves` → `cves` table (`ON CONFLICT DO UPDATE`), optional `cve_change_history` rows.
3. **Post-process:** strip auto-summaries, backfill display fields, `enrich_cves_extended` (SploitUS/CIRCL).
4. **List:** `GET /api/cves` builds SQL from `_build_cve_filters`, `paginate(page, limit max 50)`.
5. **UI:** `CVEFeed.jsx:loadPage` → `fetchCVEs` → `CVECard.jsx` renders each row.

Sequence diagram: [docs/diagrams/flow_cve_feed.mermaid](#)

B. CVE detail drill-down

1. **Card click:** `App.jsx:handleSelectCVE` sets list CVE, then `fetchCVE(cve_id)` → `GET /api/cves/{id}`.
2. **Server enrichment (serial awaits in handler):** SploitUS exploits, GreyNoise scans, OTX pulses, OSV packages, CIRCL merge (`main.py:912–971`).
3. **Drawer opens** with enriched CVE; parallel client fetches on `cve_id` change:
4. `GET /api/cves/{id}/sentences` (immediate)
5. `GET /api/cves/{id}/epss-history` (immediate)
6. `GET /api/cves/{id}/momentum` (immediate)
7. `GET /api/cves/{id}/correlation?sector=` (immediate)
8. **Lazy tab fetches:**
9. `GET /api/cves/{id}/related` — only when **Related** tab active
10. `GET /api/cves/{id}/detection` — only when **Detect** tab first opened
11. **OTX pulse IOCs:** loaded via CVE detail `otx_pulses`; pulse IOC drill-down uses `GET /api/otx/pulses/{id}/iocs`.

Known gap: `DrawerAtlasSection.jsx` expects `cve.atlas_techniques`, `cve.atlas_case_studies`, `cve.has_ai_context` but `GET /api/cves/{id}` does not call `database.get_atlas_techniques_for_cve /`

`get_atlas_case_studies_for_cve` (functions exist at `database.py:1478–1528`).

Sequence diagram: [docs/diagrams/flow_cve_detail.mermaid](#)

C. IOC lookup

1. **Input:** `IOCLookup.jsx` validates type (`ip` | `hash` | `domain`), optional GreyNoise opt-in.
2. **API:** `POST /api/ioc/lookup` → `get_ioc_cache` (6h) or `enrichment/ioc.lookup_ioc`.
3. **Per-type enrichment (sequential within shared httpx client, not `asyncio.gather`):**
4. **IP:** `VirusTotal` → `AbuseIPDB` → (optional) `GreyNoise` → `OTX`
5. **Hash:** `VirusTotal` → `MalwareBazaar`
6. **Domain:** `VirusTotal` → `URLhaus` → `OTX`
7. **Cache write:** `set_ioc_cache` with `ON CONFLICT DO UPDATE`.
8. **UI:** per-source result cards and template sentences from `templates/intelligence.py`.

Sequence diagram: [docs/diagrams/flow_ioc_lookup.mermaid](#)

D. Risk scoring (v1.1b)

Client-side (`frontend/src/scoring/riskScore.js:calculateRiskScore`):

Component	Weight
Asset profile match	0.35
KEV status	0.25
EPSS	0.15
Exploit availability	0.10
CVSS	0.10
Momentum	0.05

Momentum fetched lazily from `GET /api/cves/{id}/momentum` → `scoring/risk.py:calculate_momentum` (EPSS trend, OTX pulse recency, recent KEV, rapid exploitation signals). Cached in `momentumCache.js` for card arrows.

Display: `DetailDrawer.jsx` Overview tab — `RiskScoreBreakdown` (not Correlation tab). Cards use momentum 0 until drawer fetch updates cache.

Duplication debt: same weights/logic mirrored in `backend/scoring/risk.py` (server momentum only today).

4. Design Decisions & Trade-offs

SQLite over PostgreSQL

- **Why:** Single-user beta, zero ops overhead, `aiosqlite` async support, `feed_cache` + `ioc_cache` adequate at current scale.
- **Trade-off:** No concurrent write safety, no horizontal scaling — acceptable for v1.1.

APScheduler over Celery/Redis

- **Why:** No message broker; embedded in FastAPI process; sufficient for 7 jobs (`scheduler.py:start_scheduler`).
- **Trade-off:** Jobs lost on process restart (mitigated by `maybe_run_on_startup` bootstrap when CVE count < 10); no distributed workers.

Plain JSX + CSS over component library

- **Why:** Full control over dark terminal aesthetic; smaller bundle (`package.json` — React + Vite only).
- **Trade-off:** More custom CSS; no pre-built accessibility primitives.

Client-side risk scoring

- **Why:** Zero API calls for score on cards; instant recalculation when asset profile changes.
- **Trade-off:** Weights duplicated in Python (`scoring/risk.py`) and JavaScript (`scoring/riskScore.js`) — v1.2 will serve single config.

Monolithic `main.py` (intentional v1.1)

- **Why:** Single-developer velocity; no premature abstraction.
- **Trade-off:** 1,344 lines, ~10 responsibilities — v1.2 router + service split planned.

Monolithic `database.py` (intentional v1.1)

- **Why:** Single-file DAL easy to audit; no ORM.
 - **Trade-off:** 1,681 lines — v1.2 `repositories/` extraction planned.
-

5. System Design Principles Status

Principle	v1.1 Status	v1.2 Plan
Separation of Concerns	PARTIAL	<code>services/</code> layer (cve, enrichment, ioc, detection)
Single Responsibility	PARTIAL	Router split; <code>DetailDrawer.jsx</code> (1,516 lines) component extraction
Repository Pattern	MISSING	<code>repositories/</code> from <code>database.py</code>
Dependency Injection	MISSING	FastAPI <code>Depends()</code> for DB + <code>settings.py</code>
Circuit Breaker	PARTIAL	<code>resilient_client.py</code> wrapper (NVD has retry only today)
Idempotency	PARTIAL	Upserts + scheduler locks; fix <code>cve_change_history</code> duplicate inserts
Caching Strategy	PARTIAL	<code>feed_cache / ioc_cache</code> exist; add React Query + stats cache
API Consistency	PARTIAL	v1.2 response envelope (<code>data + meta</code>)
Config Management	PARTIAL	<code>settings.py</code> ; centralize weights and TTLs
Observability	PARTIAL	Plain logging; add request IDs + structured JSON

6. External Dependencies Map

Service	Used by	Data provided	Key env var	Free tier
NVD	<code>feeds/nvd.py</code> , scheduler	CVE records, CVSS, CPE	<code>NVD_API_KEY</code> (optional)	50 req/30s with key
CISA KEV	<code>feeds/kev.py</code>	KEV catalog JSON	—	Unrestricted

Service	Used by	Data provided	Key env var	Free tier
EPSS	<code>feeds/epss.py</code>	Exploit prediction scores	—	Unrestricted
MITRE STIX	<code>feeds/mitre.py</code>	Techniques, groups, CVE maps	—	Unrestricted
ATLAS YAML	<code>feeds/atlas.py</code>	AI/ML techniques, case studies	<code>ATLAS_YAML_URL</code>	Unrestricted
Sploitus	<code>feeds/extended.py</code>	Public exploits	—	Unpublished
GreyNoise	<code>feeds/extended.py</code> , IOC	IP classification	<code>GREYNOISE_API_KEY</code>	50/week
VirusTotal	<code>enrichment/ioc.py</code>	IP/hash/domain reputation	<code>VIRUSTOTAL_API_KEY</code>	500/day
AbuseIPDB	<code>enrichment/ioc.py</code>	IP abuse score	<code>ABUSEIPDB_API_KEY</code>	1000/day
OTX	<code>feeds/otx.py</code>	Pulses, IOCs	<code>OTX_API_KEY</code>	10k/month
OSV.dev	<code>feeds/osv.py</code>	Package affected versions	—	Unrestricted
CIRCL	<code>feeds/extended.py</code>	Extra refs, CAPEC	—	Unrestricted
MalwareBazaar	<code>feeds/extended.py</code>	Hash metadata	<code>ABUSECH_AUTH_KEY</code>	Fair use
URLhaus	<code>feeds/extended.py</code>	Domain malware URLs	<code>ABUSECH_AUTH_KEY</code>	Fair use
Groq	<code>ai/summary.py</code>	Executive summary	<code>GROQ_API_KEY</code>	Console quota

Service	Used by	Data provided	Key env var	Free tier
Anthropic	ai/summary.py	Executive summary	ANTHROPIC_API_KEY	Console quota
GitHub	detection/ rule_sources.py	Sigma/ Elastic rule search	GITHUB_TOKEN (optional)	60/hr anon
RSS (6 sources)	feeds/ incident_news.py	News cards	—	Per-feed

RSS sources defined in feeds/incident_sources.py: The Hacker News, Bleeping Computer, Krebs, Dark Reading, Schneier, CISA Advisories.

7. Known Limitations — v1.1 Beta

- **Single-user SQLite** — no concurrent write safety under heavy parallel writes.
 - **No authentication** on any /api/* endpoint.
 - **POST /api/investigation/summary broken** — calls undefined generate_investigation_summary (main.py: 1331); will 500 at runtime. Low priority; use POST /api/ai/summary instead.
 - **ATLAS per-CVE data missing from GET /api/cves/{id}** — DrawerAtlasSection never receives atlas_techniques / has_ai_context from API (P0 wiring fix in progress).
 - **Risk weights duplicated** in backend/scoring/risk.py and frontend/src/scoring/riskScore.js.
 - **No circuit breakers** on external APIs (timeouts only).
 - **DetailDrawer.jsx — 1,516 lines** — maintenance risk; v1.2 split planned.
 - **No request ID tracking** across request lifecycle.
 - **Dead code:** frontend/src/utils/riskScore.js (v1.1a, unused), Phase2Block in DetailDrawer (defined, never rendered), AIThreats.jsx (not imported).
-

8. v1.2 Refactor Roadmap

Phase	Scope
1	<code>settings.py</code> + <code>dependencies.py</code> + split <code>main.py</code> into <code>routers/</code>
2	<code>services/cve_service.py</code> , <code>services/enrichment_service.py</code> , <code>services/ioc_service.py</code>
3	<code>repositories/</code> extracted from <code>database.py</code> (1,681 lines)
4	Frontend hooks (<code>useCVEFeed</code> , <code>useCveDrawerData</code> , <code>useIOCLookup</code>) + <code>DetailDrawer</code> component split
5	Structured logging, request IDs, API response envelope, shared HTTP resilience

Related documentation

- [API_REFERENCE.md](#) — endpoint catalog
- [TECHNICAL_INVENTORY.md](#) — schema, scheduler, stack
- [APPLICATION_EXECUTION_MAP.md](#) — startup and request journeys
- [FOLDER_STRUCTURE_GUIDE.md](#) — file-by-file map
- [docs/diagrams/](#) — Mermaid diagrams (render in GitHub, VS Code, Notion)